

Guide 1 — Chef d'équipe, intégration et déploiement

Rôle : Randy. Adjoint : le rôle Sécurité & admin (il reprend tout quand le chef dort).

Ta mission n'est pas de coder le plus : c'est que **l'équipe livre au jury des fonctionnalités qui marchent en ligne**. Tu décides, tu intègres, tu déploies, tu protèges les autres des interruptions.

1. Les 30 premières minutes (H0 → H+0h30)

1. Copier **tout le sujet** (texte + doc de l'API de l'orga) et le coller à Claude avec : « Voici le sujet. Donne le modèle de données, les routes, le backlog priorisé P0/P1/P2, la direction visuelle, et les commandes `make:feature` pour les fonctionnalités de base. »
2. Pendant ce temps, les 3 autres lisent le sujet chacun de leur côté.
3. **Réunion de 10 min** : valider le modèle de données, répartir les 7 fonctionnalités de base, créer les issues.
4. Lancer les `make:feature` des entités principales **toi-même, tout de suite**, commiter, pousser. Tout le monde part de la même base.

2. Le triage d'une annonce (10 min maximum)

À chaque nouvelle fonctionnalité annoncée :

1. `/trriage <texte de l'annonce>` dans Claude Code (ou coller dans Claude).
2. Décider : **P0** (obligatoire), **P1** (forte valeur jury, effort raisonnable), **P2** (si le temps), ou **non**.
3. Créer l'issue avec le bon label et la priorité, l'assigner.
4. Annoncer la décision en une phrase sur le vocal. Pas de débat au-delà de 10 min.

Grille de décision rapide :

Question	Si oui
Le starter la couvre déjà (rôles, admin, upload, filtres) ?	P1 immédiat : points faciles
Fonctionnalité de sécurité ?	P1 : critère explicite du jury
Plus de 1h30 et touche au cœur de l'app ?	P2 ou non, sauf si c'est une fonctionnalité de base
Risque de casser ce qui marche ?	Seulement si le reste est stable

3. Le board GitHub

Colonnes : **À faire** → **En cours** → **À tester sur Hodi** → **Fait & dans le récap**.

- Chaque dev déplace sa carte en « En cours » quand il commence.
- PR mergée → la carte passe seule en « À tester sur Hodi ».

- **Toi**, après déploiement et test en ligne → tu fermes l'issue → « Fait & dans le récap ».
- Dans les PR : Refs #12 , **jamais** Closes #12 .
- La dernière colonne **est** le récap jury. Rien n'y entre sans test en ligne.

4. Intégrer (merger) le travail des autres

Avant de merger une PR :

1. La coche verte GitHub est là (Pint + tests).
2. `git pull` puis `git checkout feat/xxx` en local, `php artisan migrate` , tester la page 2 minutes.
3. Vérifier : route dans le groupe `auth` , `authorize` présent, un test « autre utilisateur → 403 ».
4. Merger, puis `git checkout main && git pull` .

En cas de conflit : si c'est dans `routes/features.php` , la sidebar ou le seeder (fichiers modifiés par le générateur), garder **les deux** blocs. Sinon, demander à l'auteur de la PR de résoudre.

5. Déployer

Sur le serveur (Terminal cPanel) :

```
bash ~/webcup-2026/deploy.sh
```

Le script fait : `git pull` , `composer install` , `npm ci` + `npm run build` , `migrate --force` , `storage:link` , `filament:assets` , `optimize` . Il s'arrête à la première erreur.

Après chaque déploiement (2 min) :

1. Ouvrir le site en navigation privée.
2. Se connecter avec le compte jury **user**, tester la fonctionnalité déployée.
3. Se connecter avec le compte jury **admin**, ouvrir `/admin` .
4. Tester sur téléphone (ou mode responsive du navigateur).

Rythme : un déploiement toutes les 2-3 h minimum. Jamais « on déploiera à la fin ».

6. Quand ça casse en production

```
cd ~/webcup-2026
tail -n 60 storage/logs/laravel.log      # la vraie erreur est ici
php84 artisan about                      # état général (env, debug, versions)
php84 artisan migrate:status              # migrations passées ?
php84 artisan optimize:clear && php84 artisan optimize  # vider les caches
```

Symptôme	Cause probable	Action
Erreur 500 partout	Cache de config ou <code>.env</code>	<code>optimize:clear</code> puis <code>optimize</code>
« PHP version >= 8.x »	Bloc AddHandler supprimé de <code>public/.htaccess</code>	Le remettre (voir CLAUDE.md)
Page sans style	Build front raté	Relancer <code>deploy.sh</code> , lire l'erreur <code>npm run build</code>
Photos invisibles	Lien <code>storage</code> absent	<code>php84 artisan storage:link</code>
Migration qui échoue	Différence SQLite / MariaDB	Lire l'erreur, corriger en local, nouvelle migration

Revenir en arrière (une fonctionnalité casse tout et on n'a pas le temps de corriger) : en local, `git revert <sha-du-commit>`, push, puis `deploy.sh`. Ne jamais faire de `git reset --hard` sur le serveur.

7. Règles absolues en production

- `APP_DEBUG=false`, toujours.
- Jamais `migrate:fresh` ni `db:seed` sur le serveur (on effacerait tout ; Laravel bloque d'ailleurs `migrate:fresh` en production).
- Jamais `key:generate` si la clé existe déjà.
- Données de démo en production : les créer **via l'application** ou via un seeder dédié, jamais en rejouant `DatabaseSeeder`.

8. Comptes jury

À créer en production dès que le socle est en ligne :

- un compte **utilisateur** (ex. `jury@webcup.com`) ;
- un compte **admin** (ex. `admin@webcup.com`), passé en admin via `/admin/users` ou :

```
php84 artisan tinker --execute="App\Models\User::where('email', 'jury-admin@webcup.com')->update(['role' => 'admin']);"
```

Mots de passe **uniques** (ils seront dans le README), 12 caractères minimum, jamais réutilisés ailleurs.

9. Le planning des 24 h

Heure	Toi
H0 → H+1	Sujet → Claude, réunion, génération des entités, répartition
H+1 → H+6	Intégration continue, socle en ligne au plus tard à H+6
H+6 → H+10	Triage des annonces, merges, déploiements toutes les 2-3 h
H+10 → H+13	L'adjoint gère (duo B dort avant toi, puis tu dors H+13 → H+16)
H+16 → H+21	Fonctionnalités progressives, déploiements
H+21	Gel : plus aucune nouvelle fonctionnalité
H+21 → H+23	/audit-secu , corrections, mobile, finitions
H+23h30	Dernier déploiement , test complet en ligne
Dimanche après-midi	Après avoir dormi : /recap-jury , vidéo, formulaire du dashboard Webcup

10. Livrables du jury (dimanche 11h → lundi 11h)

- URL de l'application, comptes jury (user + admin), lien du repo GitHub.
- Récapitulatif : **uniquement** les fonctionnalités de la colonne « Fait & dans le récap ».
- Vidéo (3 min) : tournée sur l'application **en ligne**, pas en local.
- README : installation, comptes, architecture, liste des fonctionnalités, sécurité.